

Autoresearch Loops: Causal Study of the Instruction File

Christian Block

Research project with Prof. Ramesh

August 31, 2026

1 Introduction

In an autoresearch loop, a language model does the experimenting. It is given a goal, some code it may edit, and an executor that runs a trial and hands back a number. From there it works on its own: propose a change, run it, read the result, decide what to try next, and repeat until the budget runs out. Karpathy's `autoresearch` [10] fixed the current meaning of the term when it appeared in March 2026, and it has been forked several thousand times since.

The architecture is a contract between three files, each with one owner. `prepare.py` prepares the data and defines the score. Nobody may touch it, so every experiment is scored the same way. `train.py` holds the model and the training loop, and only the agent edits it. `program.md` holds the instructions, and only the human edits it. Every candidate gets five minutes of wall clock and one number, validation bits per byte, so trials cost the same and compare directly. The loop ratchets through `git`: commit a change, train, keep the commit if the score improved, otherwise `git reset` back [2]. The repository only ever moves forward.

What the system does not contain is any orchestration code. There is no scheduler and no driver program. The nine-step experiment cycle exists only as English prose inside `program.md`, and the model is the execution layer. So `program.md` is not configuration handed to a controller. It is the controller.

1.1 Problem statement

We know these loops work. We do not know why they behave as they do. The AI Scientist [11] and Coscientist [7] are judged end to end, on whether the pipeline produced something acceptable, and benchmarks like MLAGentBench [9] report success rates over task suites. Industry reports the same shape of result. An autoresearch run against a production template engine cut parse-and-render time by 53% over about one hundred and twenty experiments, a gain its own author called probably overfitted and which was never merged [1]. All of this shows a system can work. None of it says which part of the instruction file did the work.

The obstacle is confounding. An instruction file settles several things at once: what counts as success, how widely the agent may search, when it must stop, how it must report, where to spend effort. Compare two differently worded files and every sentence differs, so nothing is identified. Work on prompt sensitivity makes the problem sharper. Formatting choices that preserve meaning can move accuracy by 76 points [12], while models sometimes do just as well on instructions that are irrelevant or actively misleading [14]. Wording matters, and reading the wording will not tell you how. That is when you run an experiment.

1.2 Related work

Three areas of work are relevant here. None of them answers the question.

The *agent architectures* that made the loop possible fixed the propose–act–observe template. ReAct interleaves reasoning traces with actions [15]. Reflexion adds a verbal self-critique the agent carries into the next trial [13]. SWE-agent supplies a restricted interface for edit-and-test cycles over a repository [6], and the Ralph technique keeps long runs stable by holding state in files instead of a context window [5]. Structurally this is a control loop of the kind autonomic computing described [4] and the Kubernetes controller implements [3]: compare an observed state against a declared one, act to close the gap, repeat forever. But there is one big difference. A controller’s logic is code you can open and read. Ours is a paragraph of English.

Attribution methods for configurable systems are mature, but they grew up around formally declared hyperparameter spaces. Functional ANOVA splits the variance of a performance measure into per-hyperparameter shares [17]. Ablation analysis walks between two configurations and charges the performance gap to the individual differences [16]. Factorial designs, classically the Plackett–Burman construction [18], supply the runs both techniques need. Once the configuration is written in English, none of this has an established counterpart.

Causal inference for text supplies the vocabulary. Feder et al. set out the cases where text is an outcome, a treatment, or a confounder [8]. The treatment case is ours: the instruction file is the treatment, the loop’s behaviour is the outcome, and the components of the file normally move together.

1.3 Objective and contribution

This project stops treating the instruction file as prose and treats it as a configuration vector. Five components of `program.md` are each written at two levels. All $2^5 = 32$ combinations are generated and run against a real executor, which separates the effect of each component instead of assuming it. The contribution is a method, not an architecture. We do not offer a better autoresearch loop. We offer a way to find out what an existing one is responding to, and we report the effects the method finds together with the ones it rules out.

The scope is narrow on purpose. Nothing here compares the loop to a human researcher. There is no human baseline and no external standard of good research practice. Every comparison is between two configurations of the same system.

RQ1 Which components of `program.md` causally determine which aspects of the loop’s research *process*, and how large is each effect?

RQ2 Do any of these components move the *outcome* the loop reaches, once the outcome is measured on a scale that is not saturated by the task, and for which of them can an effect larger than a pre-specified threshold be excluded?

RQ3 Do the components interact, and how much error does the eight-run fractional design used in earlier work incur by assuming that they do not?

Section 2 sets out the loop and its configuration space. Section 3 covers the design, the results and what they mean. Section 4 closes.

2 Background

2.1 The loop under study

Our loop follows the protocol above with one narrowing: the search space is a hyperparameter space, not arbitrary code, so that the instruction file stays the only free variable. The file arrives once as the system

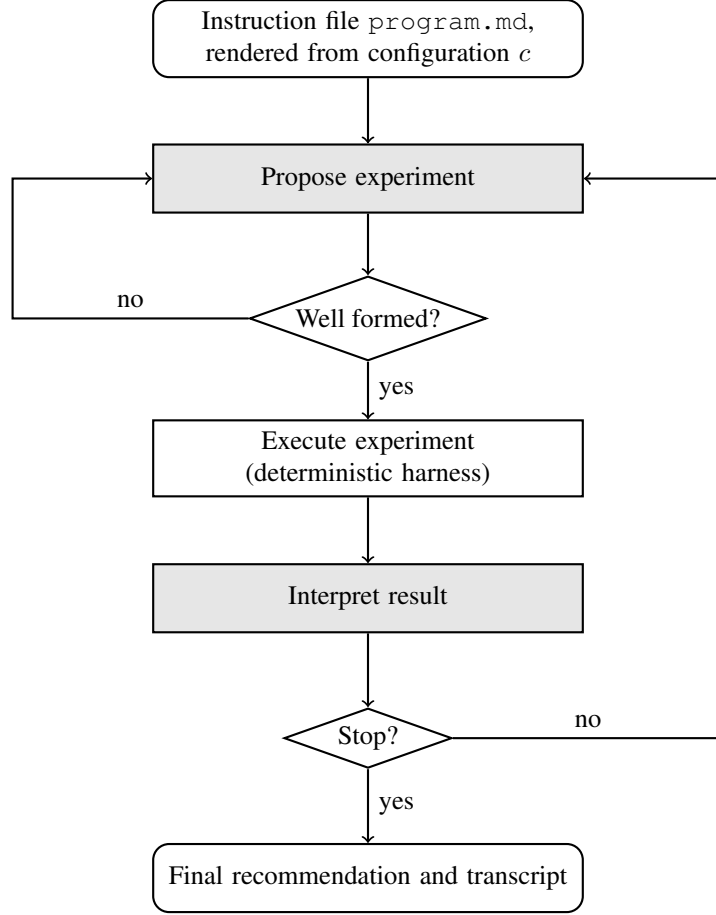


Figure 1: Control flow of one run. Shaded boxes are language-model calls. The unshaded box is deterministic, so every number the loop reasons about was measured and not generated. A malformed or rejected proposal goes back to the proposal step without spending an experiment. The loop halts on the model’s decision or on the harness budget, whichever comes first.

message and never changes after that. The model then alternates between two moves, each a single JSON object. A `propose` carries a hypothesis, a model family and hyperparameter values. An `interpret` carries a reading of the last measurement and a decision to continue or stop.

The harness does the executing. It validates each proposal against a fixed whitelist and fits the estimator with `scikit-learn`. The model never computes a number. It picks what gets computed and reads the answer back. Every value in a transcript is thus traceable: it came from the executor, making it a fact about the task, or from the model, making it a fact about the loop’s behaviour. Figure 1 shows the control flow.

2.2 The instruction file as a configuration

Every instruction file comes out of one template. The fixed part gives the research question, the executor interface and the JSON response format, and is byte-identical across all thirty-two files. The variable part is five paragraphs, each written at one of two levels. Those five slots are the independent variables. Nothing else in the file moves. A file is then described completely by

$$c = (c_1, \dots, c_5), \quad c_i \in \{0, 1\},$$

taking the slots in the order metric, breadth, stopping rule, output format, emphasis. That gives $2^5 = 32$ files.

Table 1 lists the levels. They are not invented for this study. A real `program.md` hardcodes the baseline

Table 1: The five varied components of `program.md`. A configuration is one choice of level per row.

Code	Component	Level 0	Level 1
<i>M</i>	Evaluation criterion	Success left undefined; the loop judges quality as it sees fit.	Success defined as five-fold cross-validated accuracy, with the across-fold standard deviation used only to break ties.
<i>B</i>	Search breadth	Commit to one model family for the session.	Compare both families within a session and vary several hyperparameters at once.
<i>S</i>	Stopping rule	Fixed budget of exactly three experiments.	Adaptive budget of up to eight, halted when an experiment fails to improve by more than 0.002.
<i>O</i>	Reporting style	Terse: the action taken and the numbers involved.	Full sentences stating hypothesis, expectation and interpretation.
<i>E</i>	Search emphasis	Explore first: several different configurations before refining any.	Exploit first: refine a promising configuration immediately.

to beat, says how failures should be handled, fixes a reporting discipline and imposes a parsimony rule [2], and the five components below come from those categories.

A language model writes the ten paragraphs from fixed meta-prompts. They are then frozen and reused word for word. This is a control. Had we written them ourselves, the wording of every treatment would have been a free parameter tangled up with our own stylistic habits, and no measured effect could be separated from it. The generator is a different model from the agent. Each paragraph is checked to entail its intended level and to say nothing about any other slot, and the two levels of a slot are matched to within ten per cent in token count.

2.3 Configuration change as intervention

We construct the configurations. For each of the thirty-two values of c we generate the file and run it. In observational prompt data the components would correlate, since whoever writes a precise success criterion probably also writes a careful stopping rule. Construction breaks that correlation. Across the thirty-two files each component sits at level 1 in exactly sixteen, and inside each half the other four stay balanced. The difference in mean outcome between runs with $c_i = 1$ and runs with $c_i = 0$ cannot then be produced by any other component. Confounding goes away by intervention, without any statistical adjustment.

Five main effects do not need thirty-two files. Eight will do, which is what the Plackett–Burman designs [18] and the earlier study rely on. The catch is that a fraction cannot pull interactions apart from main effects: in an eight-run design each estimate is a main effect plus a sum of interaction terms, and nothing inside the fraction can test whether that sum is small. Running all thirty-two removes the catch, turns component independence into a finding, and lets us audit the earlier study’s assumption directly. The price is thirty-two cells instead of eight. At this scale we can afford it.

3 Numerical Evaluation

3.1 Experimental setup

Tasks and executor. Every configuration runs against two canonical tabular classification datasets, picked to differ in size, dimensionality and class count: Breast Cancer Wisconsin (569×30 , two classes) and Wine (178×13 , three classes). Each uses one stratified 70/30 partition, fixed

with the same seed everywhere. The loop may reach for `RandomForestClassifier` or `GradientBoostingClassifier` under a fixed hyperparameter whitelist. Whatever a configuration’s evaluation paragraph stresses, the executor returns the same record every time: five-fold cross-validated accuracy, its across-fold standard deviation, and held-out validation accuracy. The instruction file changes what the loop attends to. It never changes what gets measured.

Baselines. Two reference points are computed per dataset before any run, by procedures that involve no language model. $a_0(d)$ is a default-hyperparameter fit. $a^*(d)$ is an exhaustive grid search over the whitelist the loop may use. They are not a bar the loop has to clear; they exist to put tasks of different difficulty on one scale. A third baseline is methodological: the eight-run fractional design from the earlier study, which Section 3.5 compares against the full design. We measured the headroom instead of assuming it. The cross-validated gap $a^* - a_0$ is 0.0100 on the first dataset and 0.0080 on the second, so both support a regret measurement. On the validation scale the second dataset has no headroom at all: its 54-instance split is classified perfectly by defaults. We report that dataset on the cross-validated scale only.

Control of non-treatment variation. These are held fixed and so cannot explain any measured effect: the agent model, pinned to the dated snapshot `gpt-4o-mini-2024-07-18` instead of a moving alias; the decoding temperature, one value for every run; the partition and its seed; the executor and its whitelist; the fixed part of the template; the response format; and the harness safety cap. Runs are separate conversations, so nothing carries over between them.

Sampling noise is different. We control it instead of removing it. Every run passes an explicit sampling seed to the model interface, so any run can be regenerated exactly, and every cell draws from the same seed list, so the draws cannot favour one configuration. Greedy decoding at temperature zero would be the wrong move here. Run-to-run variance is the error term that every coefficient below is tested against, and a deterministic loop drives it to zero, taking every standard error with it. We hold temperature constant and above zero, which makes it a controlled constant. When a run fails to produce a valid experiment it is repeated with a fresh seed until the cell holds exactly R completed runs, so balance survives by construction.

Design and budget. The design is the complete 2^5 factorial crossed with the task suite and replicated R times, giving $N = 2^5 \cdot D \cdot R$ runs. We set $D = 2$ and $R = 20$, so $N = 1280$. That replicate count is not convenience. It fixes the smallest detectable level switch at 0.22σ per dataset, about six per cent of the interval between an untuned fit and an exhaustive search.

3.2 Performance metrics

Metrics come straight off the structured transcript. No evaluation framework, scoring library or judge model is involved, because every metric is a count over declared action types, a spread over proposed values, or an accuracy the executor returned.

Measurement window. The stopping rule sets how many experiments a run performs, and several metrics aggregate over those experiments. A spread across more proposals is wider in expectation, and a best-of- n accuracy improves with n for reasons that have nothing to do with how well the loop searched. Every metric is computed over the first three experiments, the number the fixed-budget level performs and so the number every run has. Complete-run values appear separately.

Process metrics. The **number of experiments** counts executor calls. The **number of model families tried** tells us whether the breadth component’s permission was ever used. The **search width** is the spread of values tried for the hyperparameter both families share, normalised by its admissible range.

The **refinement rate** is the share of proposals that perturb their predecessor locally, this is how explore-versus-exploit shows up in the transcript.

Outcome metrics. Raw accuracy does not work across tasks of different difficulty, and a task at its ceiling looks exactly like a component with no effect. The primary outcome is the **normalised regret**

$$\text{reg}(\tau, d) = \frac{a^*(d) - a_{\text{best}}(\tau)}{a^*(d) - a_0(d)}, \quad (1)$$

where $\text{reg} = 0$ means the loop matched an exhaustive search and $\text{reg} = 1$ means it did no better than defaults. Negative values are kept rather than clipped, since clipping shifts configuration means by different amounts. The secondary outcome is **regret per executor call**. One thing to watch: the evaluation-criterion component at level 1 names the exact quantity regret is built from, so part of any effect it shows is just the treatment agreeing with the instrument. We compute regret on the cross-validated and the validation scale both, and claim an effect for that component only if it survives on each.

Instrumentation metric. The **mean length of stated hypotheses** is a crude verbosity proxy and holds no interest of its own. We include it because we already know what the answer should be: the reporting-style component tells the loop directly how much to write. If the estimation fails to name that component as the driver, the pipeline is broken and the rest of the results cannot be trusted. Four components have such a counterpart. The evaluation criterion has none by design, since it prescribes no action, so a null main effect there says nothing about the instrument.

3.3 Regression model and inference

Code each component as an indicator variable

$$X_i = \begin{cases} +1 & \text{if component } i \text{ is at level 1,} \\ -1 & \text{if at level 0,} \end{cases} \quad i = 1, \dots, 5, \quad (2)$$

and model the outcome of one run as

$$Y = \beta_0 + \sum_{i=1}^5 \beta_i X_i + \varepsilon, \quad (3)$$

with β_0 the mean across configurations, β_i the effect of component i , and ε run-to-run error. One regression per metric per dataset. Each X_i takes each value equally often and independently of the rest, so $\hat{\beta}_i$ does not shift when another component enters or leaves the model, and $2\beta_i$ is the average change in Y from switching component i . Running all thirty-two configurations buys the interaction model for free,

$$Y = \beta_0 + \sum_{i=1}^5 \beta_i X_i + \sum_{1 \leq i < j \leq 5} \beta_{ij} X_i X_j + \varepsilon, \quad (4)$$

which is how RQ3 gets tested.

We fit by ordinary least squares on the individual runs, not the cell means. Balance means the coefficients come out the same either way, while the error term now reflects real run-to-run scatter. Balance would also equalise the standard errors if the error variance were constant, and it is not: the stopping rule pins the experiment count at one level and frees it at the other, so within-cell variance differs systematically between the two halves of the design. We use heteroskedasticity-consistent standard errors throughout, and fit two metrics on their natural scale, the family count being binary and the refinement rate a proportion. Benjamini–Hochberg adjustment handles multiplicity, with the family taken as the coefficients of one model on one metric and dataset. Claiming a component has *no* effect takes more than failing to find one, so we fix a smallest effect of interest in advance and claim absence only when a two-one-sided-test procedure puts the whole confidence interval for $2\beta_i$ inside it.

Table 2: Main-effect estimates $2\hat{\beta}_i$ per metric, with heteroskedasticity-consistent standard errors and Benjamini–Hochberg adjusted p -values. **[PLACEHOLDER: populate from regression table.csv.]**

Metric	M	B	S	O	E	R^2
Number of experiments	–	–	–	–	–	–
Model families tried	–	–	–	–	–	–
Search width	–	–	–	–	–	–
Refinement rate	–	–	–	–	–	–
Normalised regret	–	–	–	–	–	–
Regret per executor call	–	–	–	–	–	–
Verbosity (instrumentation)	–	–	–	–	–	–

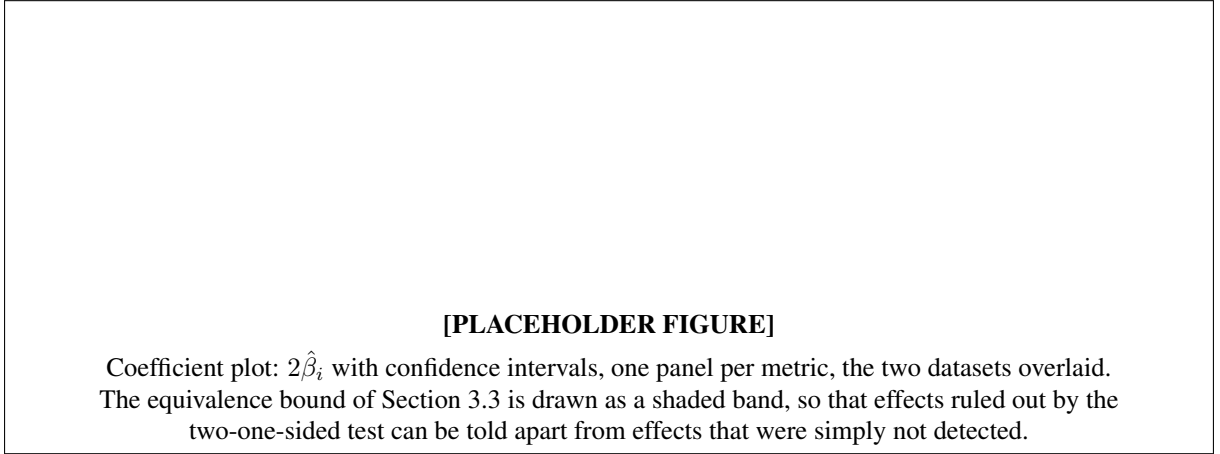


Figure 2: Estimated component effects on every metric. **[PLACEHOLDER.]**

3.4 Results

Table 2 gives the main-effect estimates and Figure 2 plots them with their intervals. **[PLACEHOLDER: for each process metric, say which component dominates and by how much, and confirm that the instrumentation metric comes out driven by reporting style as required.]**

Table 3 gives the two-way interactions. **[PLACEHOLDER: say whether the components act independently, and name any pair whose interaction rivals a main effect in size.]**

An instruction is not always obeyed. Table 3 therefore also reports, per component, the share of runs whose behaviour actually matched the instruction given, so we can tell a component that had no effect from one the model ignored. **[PLACEHOLDER.]**

3.5 Validation

A model fitted on the data used to build it has shown association and nothing more. Exhaustive enumeration leaves no unrun configuration to hold out, so we use three substitutes.

Leave-one-cell-out prediction. Refit the model thirty-two times, each time dropping all replicates of one configuration and predicting that cell’s mean before looking at its data. Aggregating gives an out-of-sample R^2 per metric, which separates the metrics the design predicts from the ones it merely describes.

Comparison of the per-dataset fits. Compare the two coefficient vectors component by component. This is a comparison, and we do not call it a test. With two tasks, holding one out leaves a single fold, far too little to say how an effect varies across tasks. Disagreement is the useful outcome, since it pins an effect to a task. Agreement cannot establish the converse.

Table 3: Two-way interaction estimates $2\hat{\beta}_{ij}$ from equation (4), and compliance rates per component. [PLACEHOLDER.]

Interaction	Estimate	adj. p	Component	Compliance
$M \times B$	–	–	M	–
$M \times S$	–	–	B	–
$B \times S$	–	–	S	–
$S \times E$	–	–	O	–
...	–	–	E	–

Table 4: Validation summary: out-of-sample R^2 per metric, agreement of the per-dataset coefficient vectors, and the gap between the eight-run fractional estimates and the full design. [PLACEHOLDER.]

Metric	LOCO R^2	Cross-dataset agreement	Fractional-design error
Number of experiments	–	–	–
Model families tried	–	–	–
Search width	–	–	–
Refinement rate	–	–	–
Normalised regret	–	–	–

Audit of the fractional design. Pull an eight-run Plackett–Burman subset out of the thirty-two cells, fit a main-effects model to it exactly as the earlier study did, and compare its coefficients with the full design. The alias structure of a resolution-III design is known analytically, so the direction of the discrepancy is no discovery. What the audit measures is how big the absorbed interaction terms actually are, and therefore how much error the sparsity assumption costs in this problem class. That answers RQ3, and it is the one check here that helps anyone who cannot afford full enumeration.

3.6 Discussion and further considerations

[PLACEHOLDER: read the recovered effect structure. Say which components govern process and whether any governs outcome; tie a null outcome result to the equivalence bound instead of to a failed significance test; and quantify what the fractional design would have got wrong.]

Three limitations bound what any of this shows. Everything is conditional on one agent model, so we have measured how instructions act on one policy, not a property of instructions at large. Each level is realised by a single generated paragraph, which leaves the effect of a component’s level tangled with the effect of its particular wording; read the results as effects of specific instruction texts. And the tasks are tabular classification problems over a small discrete hyperparameter space. Whether the same effect structure appears in loops that search over code, or over open-ended hypotheses, is outside what this design can reach.

4 Conclusion

In an autoresearch loop, `program.md` does the job that controller code does in ordinary software, but you cannot read it the way you read code. So we treated it as a configuration vector. Five components of `program.md` were written at two levels each, all $2^5 = 32$ configurations were generated and run against a real executor on two datasets with twenty replicates apiece, and the resulting behaviour was reduced to process, outcome and instrumentation metrics and regressed on an orthogonal ± 1 coding of the components. Because we built the configurations instead of collecting them, the coefficients can be read causally. Because the factorial is complete, whether the components interact is something we can test rather than assume.

[PLACEHOLDER: one paragraph on which components drove which behaviour, which outcome effects were ruled out at the pre-specified threshold, and the measured error of the fractional alternative.]

The method is not tied to the five components we happened to vary. Any instruction file that can be cut into slots can be studied the same way, and full enumeration stays cheap while the number of slots is small. Two follow-ups are worth doing. Sampling several wordings per level would separate a component from its phrasing, which is the sharpest limitation above. Running the procedure on a loop that edits code, not hyperparameters, would test it where the instruction file carries far more of the system’s behaviour.

References

- [1] T. Lütke. Performance: 53% faster parse+render, 61% fewer allocations. Pull request #2056, Shopify/liquid, 2026. Produced by an autoresearch run of approximately 120 experiments over 93 commits; unmerged at the time of writing. <https://github.com/Shopify/liquid/pull/2056>.
- [2] B. Tsuchiev. A guide to Andrej Karpathy’s AutoResearch: automating ML with AI agents. DataCamp tutorial, 23 March 2026. Secondary source; it reports the contents of the repository’s program.md and results log. <https://www.datacamp.com/tutorial/guide-to-autoresearch>.
- [3] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes. Borg, Omega, and Kubernetes: lessons learned from three container-management systems over a decade. *ACM Queue*, 14(1):70–93, 2016. Also *Communications of the ACM*, 59(5):50–57, 2016.
- [4] J. O. Kephart and D. M. Chess. The vision of autonomic computing. *IEEE Computer*, 36(1):41–50, 2003.
- [5] G. Huntley. The Ralph technique: running a coding agent in an autonomous shell loop with a fresh context window per iteration, 2025. Practitioner write-up; not peer reviewed. **[TODO: insert the exact post URL and access date.]**
- [6] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024. arXiv:2405.15793.
- [7] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes. Autonomous chemical research with large language models. *Nature*, 624(7992):570–578, 2023.
- [8] A. Feder, K. A. Keith, E. Manzoor, R. Pryzant, D. Sridhar, Z. Wood-Doughty, J. Eisenstein, J. Grimmer, R. Reichart, M. E. Roberts, B. M. Stewart, V. Veitch, and D. Yang. Causal inference in natural language processing: Estimation, prediction, interpretation and beyond. *Transactions of the Association for Computational Linguistics*, 10:1138–1158, 2022. Preprint arXiv:2109.00725, 2021.
- [9] Q. Huang, J. Vora, P. Liang, and J. Leskovec. MLAGentBench: Evaluating language agents on machine learning experimentation. *ICML*, 2024. arXiv:2310.03302.
- [10] A. Karpathy. autoresearch: AI agents running research on single-GPU nanochat training automatically. GitHub repository, 2026. <https://github.com/karpathy/autoresearch>.
- [11] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha. The AI Scientist: Towards fully automated open-ended scientific discovery, 2024. arXiv:2408.06292.

- [12] M. Sclar, Y. Choi, Y. Tsvetkov, and A. Suhr. Quantifying language models’ sensitivity to spurious features in prompt design, or: How I learned to start worrying about prompt formatting. ICLR, 2024. arXiv:2310.11324.
- [13] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. NeurIPS, 2023. arXiv:2303.11366.
- [14] A. Webson and E. Pavlick. Do prompt-based models really understand the meaning of their prompts? NAACL-HLT, pages 2300–2344, 2022.
- [15] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. ICLR, 2023. arXiv:2210.03629.
- [16] C. Fawcett and H. H. Hoos. Analysing differences between algorithm configurations through ablation. *Journal of Heuristics*, 22(4):431–458, 2016.
- [17] F. Hutter, H. Hoos, and K. Leyton-Brown. An efficient approach for assessing hyperparameter importance. ICML, 2014.
- [18] R. L. Plackett and J. P. Burman. The design of optimum multifactorial experiments. *Biometrika*, 33(4):305–325, 1946.